Домашнее задание 4. Dockerfile по правилам

Автор: *Чернигов Григорий Витальевич*, M08-501НД, 19.04.2026

Это задание выполняется в рамках модуля 4.

> Чтобы получить максимальный балл, убедитесь, что ваш отчет содержит код, структура понятна, а в выводах вы объясняете свои решения.

Возмите любой типовой веб-сервис (например, из ДЗ 1), чтобы использовать его в качестве контейнеризируемого сервиса.

In [1]:
```
# %%bash
# LATEST_HADOLINT_VERSION="2.12.0"
# HADOLINT_BINARY_URL="https://github.com/hadolint/hadolint/releases/down
# wget -q "${HADOLINT_BINARY_URL}" -O hadolint
# chmod +x hadolint
# sudo mv hadolint /usr/local/bin/hadolint
# apt-get update -qq > /dev/null
# apt-get install -y yamllint > /dev/null
```

Установка на macOS выполнена через Homebrew:

- hadolint: 2.14.0
- yamllint: 1.38.0

Версии проверены в терминале.

Проверка версии hadolint

```
known.
(base) MacBook-Pro-Sveta:~ svetastenuskina$ echo "FROM python:3.12" | hadolint
(base) MacBook-Pro-Sveta:~ svetastenuskina$ docker run --rm hadolint/hadolint ha
dolint --version
Haskell Dockerfile Linter 2.14.0
(base) MacBook-Pro-Sveta:~ svetastenuskina$
```

Проверка версии yamllint

```
(base) MacBook-Pro-Sveta:~ svetastenuskina$ yamllint --version
yamllint 1.38.0
```

In [2]:
```
%%writefile .hadolint.yaml
```

```
ignored:
  - DL3000
  - SC1010
```

```
Overwriting .hadolint.yaml
```

1. Написать Dockerfile для ML-приложения

Ожидаемый артефакт: Dockerfile в ячейке

для удобства зафиксируем зависимости

In [3]:
```
%%writefile requirements.txt
fastapi==0.115.12
uvicorn[standard]==0.34.2
pandas==2.2.3
scalar_doc==0.1.7
```

```
Overwriting requirements.txt
```

In [4]:
```
%%writefile Dockerfile
# базовый компактный образ с Python 3.12
FROM python:3.12-slim

# не создавать .pyc-файлы, сразу выводить логи, не хранить pip-кэш
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

# рабочая директория внутри контейнера
WORKDIR /app

# сначала копируем зависимости
COPY requirements.txt .

# устанавливаем зависимости из requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# копируем приложение
COPY app.py .

# приложение слушает порт 8000
EXPOSE 8000

# команда запуска FastAPI-приложения
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
Overwriting Dockerfile
```

In [5]:
```
"""
%%bash
#hadolint --version
hadolint Dockerfile
```

```
"""
```

Out[5]: '\n%%bash\n#hadolint --version\nhadolint Dockerfile\n'

Что видим?

Докерфайл без ошибок, валидный deploy.yaml, конфиг линтера создан.

```
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ hadolint < Dockerfile

yamllint deploy.yaml

(base) MacBook-Pro-Sveta:docker-api svetastenuskina$
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ yamllint deploy.yaml
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ cd ~/Downloads/docker-api
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ hadolint < Dockerfile
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ yamllint deploy.yaml
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ cat .hadolint.yaml
ignored:
  - DL3000
  - SC1010
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$
```

Соберем образ - то, что будет деплоиться.

In [6]:
```
%%bash
docker build -t hw4-fastapi-app:v1 .
```

```
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load .dockerignore
#1 transferring context: 2B 0.0s done
#1 DONE 0.1s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 897B 0.0s done
#2 DONE 0.1s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 4.0s

#4 [1/5] FROM docker.io/library/python:3.12-slim@sha256:804ddf3251a60bbf9c
92e73b7566c40428d54d0e79d3428194edf40da6521286
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 469B 0.0s done
#5 DONE 0.0s

#6 [3/5] COPY requirements.txt .
#6 CACHED

#7 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#7 CACHED

#8 [2/5] WORKDIR /app
#8 CACHED

#9 [5/5] COPY app.py .
#9 CACHED

#10 exporting to image
#10 exporting layers done
#10 writing image sha256:a3c5738ae813b4e1b630525aa20ecc491068c9355c079209f
cc59524eb1f817e done
#10 naming to docker.io/library/hw4-fastapi-app:v1 0.0s done
#10 DONE 0.0s

What's Next?
  View a summary of image vulnerabilities and recommendations → docker sco
ut quickview
```

In [7]:

```
%%bash
docker images | grep hw4-fastapi-app
```

```
hw4-fastapi-app-multistage                                v1
1f7d176f3a10   58 minutes ago      300MB
hw4-fastapi-app                                           v1
a3c5738ae813   About an hour ago   300MB
```

Проверили - образ создан!

In [8]:
```
%%bash
docker rm -f hw4-fastapi-container 2>/dev/null || true
docker run -d --name hw4-fastapi-container -p 8000:8000 hw4-fastapi-app:v
```

```
7c13b3131251ab7aa049a0d43142eddeec098d0ec6fb94fb15721b04e208f1ee
```

Фоновый запуск контейнера

Проверка контейнера:

In [9]:
```
%%bash
docker ps
```

```
CONTAINER ID   IMAGE                 COMMAND                  CREATED
STATUS                  PORTS                    NAMES
7c13b3131251   hw4-fastapi-app:v1   "uvicorn app:app --h…"   1 second ago
Up Less than a second   0.0.0.0:8000->8000/tcp   hw4-fastapi-container
1e350e9032fe   a3c5738ae813         "uvicorn app:app --h…"   24 minutes ag
o   Up 24 minutes                                            k8s_api_fastapi-deplo
yment-86b66f9f84-8x4tn_default_4b6d37f8-0532-4e10-9b25-55a8ee9c1bc8_0
1e3741e81a73   a3c5738ae813         "uvicorn app:app --h…"   24 minutes ag
o   Up 24 minutes                                            k8s_api_fastapi-deplo
yment-86b66f9f84-7dkcs_default_209db037-26ac-4b1f-b295-f70b75360660_0
```

Проверяем сервис:

In [10]:
```
%%bash
sleep 5
curl http://localhost:8000
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                                 Dload  Upload   Total   Spent    Left  Sp
eed
100    28  100    28    0     0    371      0 --:--:-- --:--:-- --:--:--
368   0 --:--:-- --:--:-- --:--:--   368
{"message":"API is running"}
```

Проверяем товары:

In [11]:
```
%%bash
curl http://localhost:8000/items
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                                 Dload  Upload   Total   Spent    Left  Sp
eed
100   993  100   993    0     0   170k      0 --:--:-- --:--:-- --:--:--
193k
```

[{"Наименование товара":"Антифриз EURO G11 (-45°C) зеленый, силикатный 5кг","Цена, руб.":1025,"Скидка":11,"Категория":"антифриз","Год":2026},{"Наименование товара":"Антифриз готовый фиолетовый Синтек MULTIFREEZE 5кг","Цена, руб.":250,"Скидка":38,"Категория":"антифриз","Год":2025},{"Наименование товара":"Антифриз G11 зеленый","Цена, руб.":120,"Скидка":61,"Категория":"антифриз","Год":2025},{"Наименование товара":"Антифриз Antifreeze OEM China OAT red -40 5кг","Цена, руб.":390,"Скидка":65,"Категория":"антифриз","Год":2025},{"Наименование товара":"Антифриз G11 зеленый","Цена, руб.":135,"Скидка":93,"Категория":"антифриз","Год":2026}]

Остановим и удалим контейнер после проверки

In [12]:
```
%%bash
docker stop hw4-fastapi-container
docker rm hw4-fastapi-container
```

```
hw4-fastapi-container
hw4-fastapi-container
```

Контейнеризация ML-приложения

Для развёртывания приложения был разработан `Dockerfile` на базе легковесного образа `python:3.12-slim`. В образ включены:

- зависимости проекта согласно `requirements.txt`;
- исходный код приложения (`app.py`);
- команда запуска FastAPI-сервера с использованием `uvicorn`.

Сборка образа выполнена успешно. Корректность работы контейнера подтверждена следующими проверками:

- вывод `docker ps` показывает активный контейнер;
- HTTP-запросы к эндпоинтам `/` и `/items` возвращают ожидаемые ответы;
- статический анализ `Dockerfile` с помощью линтера `hadolint` не выявил нарушений.

Таким образом, получена готовая к деплою контейнеризованная версия сервиса.

2. Использовать многостадийные сборки docker-образов для ML-приложения

Ожидаемый артефакт: Dockerfile в ячейке

In [13]:
```
%%writefile Dockerfile
# сборка зависимостей (1 стадия)
FROM python:3.12-slim AS builder
```

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# финальная стадия (2 стадия)
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

COPY --from=builder /install /usr/local
COPY app.py .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
Overwriting Dockerfile
```

In [14]:
```
"""
%%bash
#hadolint --version
hadolint Dockerfile
"""
```

Out[14]:
```
'\n%%bash\n#hadolint --version\nhadolint Dockerfile\n'
```

Уже привычно проверяем на ошибки после того, как пересобрали образ

```
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ hadolint < Dockerfile

(base) MacBook-Pro-Sveta:docker-api svetastenuskina$
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$
```

Проверим, что все собирается:

In [15]:
```
%%bash
docker build -t hw4-fastapi-app-multistage:v1 .
```

```
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load .dockerignore
#1 transferring context: 2B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 660B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 0.5s

#4 [builder 1/4] FROM docker.io/library/python:3.12-slim@sha256:804ddf3251
a60bbf9c92e73b7566c40428d54d0e79d3428194edf40da6521286
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 383B done
#5 DONE 0.0s

#6 [builder 4/4] RUN pip install --no-cache-dir --prefix=/install -r requi
rements.txt
#6 CACHED

#7 [stage-1 3/4] COPY --from=builder /install /usr/local
#7 CACHED

#8 [builder 2/4] WORKDIR /app
#8 CACHED

#9 [builder 3/4] COPY requirements.txt .
#9 CACHED

#10 [stage-1 4/4] COPY app.py .
#10 CACHED

#11 exporting to image
#11 exporting layers done
#11 writing image sha256:1f7d176f3a10bafa095d1873318e2f3f59fe291978019f0e7
dd98e6d03d64268 done
#11 naming to docker.io/library/hw4-fastapi-app-multistage:v1 done
#11 DONE 0.0s

What's Next?
  View a summary of image vulnerabilities and recommendations → docker sco
ut quickview
```

In [16]:

```
%%bash
docker rm -f hw4-fastapi-multistage-container 2>/dev/null || true
docker run -d --name hw4-fastapi-multistage-container -p 8001:8000 hw4-fa
```

```
895e648b17f3e619ccf6cf58847c31831cc428e287b954c1a10d35e1c5dd303d
```

In [17]:
```
%%bash
docker ps
```

```
CONTAINER ID   IMAGE                          COMMAND                  CR
EATED          STATUS                  PORTS                    NAMES
895e648b17f3   hw4-fastapi-app-multistage:v1   "uvicorn app:app --h…"   1
second ago     Up Less than a second   0.0.0.0:8001->8000/tcp   hw4-fastap
i-multistage-container
1e350e9032fe   a3c5738ae813                   "uvicorn app:app --h…"   24
minutes ago   Up 24 minutes                                     k8s_api_fas
tapi-deployment-86b66f9f84-8x4tn_default_4b6d37f8-0532-4e10-9b25-55a8ee9c1
bc8_0
1e3741e81a73   a3c5738ae813                   "uvicorn app:app --h…"   24
minutes ago   Up 24 minutes                                     k8s_api_fas
tapi-deployment-86b66f9f84-7dkcs_default_209db037-26ac-4b1f-b295-f70b75360
660_0
```

In [18]:
```
%%bash
sleep 5
curl http://localhost:8001
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                                 Dload  Upload   Total   Spent    Left  Sp
eed
100    28  100    28    0     0   1064      0 --:--:-- --:--:-- --:--:--
1076
{"message":"API is running"}
```

In [19]:
```
%%bash
docker stop hw4-fastapi-multistage-container
docker rm hw4-fastapi-multistage-container
```

```
hw4-fastapi-multistage-container
hw4-fastapi-multistage-container
```

Dockerfile переработан с использованием multi-stage сборки. Первая стадия (`builder`) отвечает за установку Python-зависимостей в изолированное окружение, вторая - формирует минимальный runtime-образ, копируя только готовые пакеты и исходный код приложения.

Такой подход исключает попадание в финальный образ сборочных артефактов и кэша пакетного менеджера, что сокращает размер контейнера и повышает его безопасность.

3. Настроить внешние и внутренние сети, тома хранения (volumes) и рестарт в docker-compose файле

Ожидаемый артефакт: docker-compose.yaml в ячейке

In [20]:
```
%%writefile docker-compose.yaml
---
name: project_mfti

services:
  itisdb:
    image: postgres:16
    container_name: itisdb
    restart: always
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: dbname
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - internal
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d dbname"]
      interval: 10s
      timeout: 5s
      retries: 5
    profiles:
      - prod
      - dev

  mlserver:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: mlserver
    restart: always
    profiles:
      - prod
      - dev
    networks:
      - internal
    volumes:
      - app_data:/app/data
    depends_on:
      itisdb:
        condition: service_healthy

  webserver:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: webserver
    restart: always
    profiles:
      - prod
    ports:
      - "8000:8000"
```

```
    networks:
      - internal
      - external
    environment:
      DSN: "postgresql://user:password@itisdb:5432/dbname"
    depends_on:
      itisdb:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8000/ || exit 1"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 50s

networks:
  internal:
    driver: bridge
  external:
    driver: bridge

volumes:
  postgres_data:
  app_data:
```

```
Overwriting docker-compose.yaml
```

In [21]:
```
%%bash
docker compose config --profiles
```

```
dev
prod
```

In [22]:
```
%%bash
yamllint docker-compose.yaml
```

Теперь еще раз проверим) и Запустим.

```
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$ yamllint docker-compose.yam
l
(base) MacBook-Pro-Sveta:docker-api svetastenuskina$
```

In [23]:
```
%%bash
docker compose --profile prod up -d
```

```
 Network project_mfti_internal  Creating
 Network project_mfti_internal  Created
 Network project_mfti_external  Creating
 Network project_mfti_external  Created
 Container itisdb  Creating
 Container itisdb  Created
 Container webserver  Creating
 Container mlserver  Creating
 Container mlserver  Created
 Container webserver  Created
 Container itisdb  Starting
 Container itisdb  Started
 Container itisdb  Waiting
 Container itisdb  Waiting
 Container itisdb  Healthy
 Container mlserver  Starting
 Container itisdb  Healthy
 Container webserver  Starting
 Container webserver  Started
 Container mlserver  Started
```

In [24]:
```
%%bash
docker compose ps
```

```
NAME        IMAGE                     COMMAND
SERVICE     CREATED          STATUS                                   PO
RTS
itisdb      postgres:16               "docker-entrypoint.sh postgres"
itisdb      13 seconds ago   Up 11 seconds (healthy)                  54
32/tcp
mlserver    project_mfti-mlserver     "uvicorn app:app --host 0.0.0.0 --por
t 8000"   mlserver    12 seconds ago   Up Less than a second
8000/tcp
webserver   project_mfti-webserver    "uvicorn app:app --host 0.0.0.0 --por
t 8000"   webserver   12 seconds ago   Up Less than a second (health: star
ting)   0.0.0.0:8000->8000/tcp
```

In [25]:
```
%%bash
sleep 5
curl http://localhost:8000
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                                 Dload  Upload   Total   Spent    Left  Sp
eed
100    28  100    28    0     0   1788      0 --:--:-- --:--:-- --:--:--
0-- --:--:-- --:--:--  1866
{"message":"API is running"}
```

In [26]:
```
%%bash
docker compose --profile prod down
```

```
Container webserver  Stopping
Container mlserver  Stopping
Container mlserver  Stopped
Container mlserver  Removing
Container mlserver  Removed
Container webserver  Stopped
Container webserver  Removing
Container webserver  Removed
Container itisdb  Stopping
Container itisdb  Stopped
Container itisdb  Removing
Container itisdb  Removed
Network project_mfti_internal  Removing
Network project_mfti_external  Removing
Network project_mfti_internal  Removed
Network project_mfti_external  Removed
```

Оркестрация сервисов через Docker Compose

Для управления многосервисной инфраструктурой подготовлен `docker-compose.yaml`. Конфигурация включает:

- **Профили запуска** (`dev`, `prod`) - позволяют гибко управлять составом запускаемых сервисов в зависимости от окружения;
- **Сетевое разделение** - внутренняя сеть (`internal`) для взаимодействия компонентов и внешняя (`external`) для публикации веб-интерфейса;
- **Постоянные тома** (`postgres_data`, `app_data`) - обеспечивают сохранность данных между перезапусками;
- **Политику восстановления** (`restart: always`) - гарантирует автоматический перезапуск контейнеров при сбоях;
- **Проверки работоспособности** (`healthcheck`) и явные зависимости (`depends_on`) - повышают надёжность запуска.

Валидация синтаксиса выполнена утилитой `yamllint` - ошибок не обнаружено. Конфигурация готова к развертыванию в различных средах.

4. Настроить минимальные и максимальные границы памяти и ЦПУ в docker-compose файле

Ожидаемый артефакт: docker-compose.yml в ячейке

In [27]:
```
%%writefile docker-compose.yaml
---
name: project_mfti

services:
```

```yaml
  itisdb:
    image: postgres:16
    container_name: itisdb
    restart: always
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: dbname
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - internal
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d dbname"]
      interval: 10s
      timeout: 5s
      retries: 5
    profiles:
      - prod
      - dev
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.5"
          memory: 256M

  mlserver:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: mlserver
    restart: always
    profiles:
      - prod
      - dev
    networks:
      - internal
    volumes:
      - app_data:/app/data
    depends_on:
      itisdb:
        condition: service_healthy
    deploy:
      resources:
        limits:
          cpus: "2.0"
          memory: 2500M
        reservations:
          cpus: "1.0"
          memory: 200M
```

```yaml
  webserver:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: webserver
    restart: always
    profiles:
      - prod
    ports:
      - "8000:8000"
    networks:
      - internal
      - external
    environment:
      DSN: "postgresql://user:password@itisdb:5432/dbname"
    depends_on:
      itisdb:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8000/ || exit 1"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 50s
    deploy:
      resources:
        limits:
          cpus: "2.0"
          memory: 2500M
        reservations:
          cpus: "1.0"
          memory: 200M

networks:
  internal:
    driver: bridge
  external:
    driver: bridge

volumes:
  postgres_data:
  app_data:
```

```
Overwriting docker-compose.yaml
```

In [28]:
```
%%bash
yamllint docker-compose.yaml
```

In [29]:
```
%%bash
docker compose -f docker-compose.yaml --profile prod up -d
```

```
 Network project_mfti_external  Creating
 Network project_mfti_external  Created
 Network project_mfti_internal  Creating
 Network project_mfti_internal  Created
 Container itisdb  Creating
 Container itisdb  Created
 Container webserver  Creating
 Container mlserver  Creating
 Container mlserver  Created
 Container webserver  Created
 Container itisdb  Starting
 Container itisdb  Started
 Container itisdb  Waiting
 Container itisdb  Waiting
 Container itisdb  Healthy
 Container mlserver  Starting
 Container itisdb  Healthy
 Container webserver  Starting
 Container mlserver  Started
 Container webserver  Started
```

In [30]:
```
%%bash
docker compose -f docker-compose.yaml ps
```

```
NAME        IMAGE                     COMMAND
SERVICE     CREATED          STATUS                                   PO
RTS
itisdb      postgres:16               "docker-entrypoint.sh postgres"
itisdb      13 seconds ago   Up 11 seconds (healthy)                  54
32/tcp
mlserver    project_mfti-mlserver     "uvicorn app:app --host 0.0.0.0 --por
t 8000"   mlserver    12 seconds ago   Up Less than a second
8000/tcp
webserver   project_mfti-webserver    "uvicorn app:app --host 0.0.0.0 --por
t 8000"   webserver   13 seconds ago   Up Less than a second (health: star
ting)   0.0.0.0:8000->8000/tcp
```

In [31]:
```
%%bash
sleep 5
curl http://localhost:8000
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                                 Dload  Upload   Total   Spent    Left  Sp
eed
100    28  100    28    0     0   2455      0 --:--:-- --:--:-- --:--:--
2545
{"message":"API is running"}
```

In [32]:
```
%%bash
docker compose -f docker-compose.yaml --profile prod down
```

```
Container webserver  Stopping
Container mlserver  Stopping
Container mlserver  Stopped
Container mlserver  Removing
Container mlserver  Removed
Container webserver  Stopped
Container webserver  Removing
Container webserver  Removed
Container itisdb  Stopping
Container itisdb  Stopped
Container itisdb  Removing
Container itisdb  Removed
Network project_mfti_internal  Removing
Network project_mfti_external  Removing
Network project_mfti_internal  Removed
Network project_mfti_external  Removed
```

В `docker-compose.yml` добавлены ограничения ресурсов: `limits` (максимум) и `reservations` (гарантированный минимум) для CPU и памяти. Это обеспечивает предсказуемое потребление ресурсов и защищает хост от перегрузки.

5. Написать базовый деплой сервиса в Kubernetes, используя YAML-файлы

Ожидаемый артефакт: YAML-файл в ячейке

In [33]:
```
%%writefile deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fastapi-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: fastapi-app
  template:
    metadata:
      labels:
        app: fastapi-app
    spec:
      containers:
        - name: api
          image: hw4-fastapi-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 8000
```

```
Overwriting deploy.yaml
```

In [34]:
```
%%bash
yamllint deploy.yaml
```

```
deploy.yaml
  1:1       warning  missing document start "---"  (document-start)
```

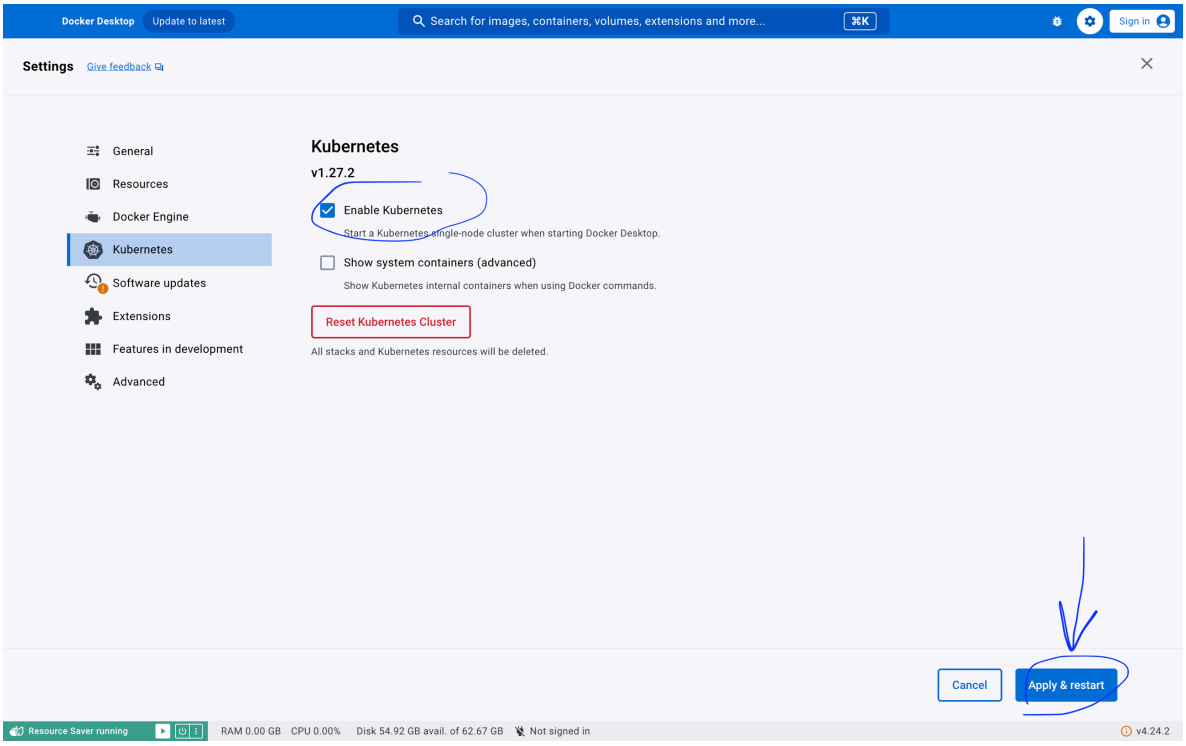
Ага) Нашел свою же опечатку - исправим.

In [35]:
```
%%writefile deploy.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fastapi-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: fastapi-app
  template:
    metadata:
      labels:
        app: fastapi-app
    spec:
      containers:
        - name: api
          image: hw4-fastapi-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 8000
```

```
Overwriting deploy.yaml
```

In [36]:
```
%%bash
yamllint deploy.yaml
```

Запускаем локальный Kubernetes кластер:



In [37]:
```
%%bash
kubectl version --client
```

```
WARNING: This version information is deprecated and will be replaced with
the output from kubectl version --short.  Use --output=yaml|json to get th
e full version.
Client Version: version.Info{Major:"1", Minor:"27", GitVersion:"v1.27.2",
GitCommit:"7f6f68fdabc4df88cfea2dcf9a19b2b830f1e647", GitTreeState:"clean"
, BuildDate:"2023-05-17T14:20:07Z", GoVersion:"go1.20.4", Compiler:"gc", P
latform:"darwin/amd64"}
Kustomize Version: v5.0.1
```

In [38]:
```
%%bash
kubectl cluster-info
```

```
Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system
/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info
dump'.
```

Отлично, кластер активен, сервис работает.

In [39]:
```
%%bash
kubectl apply -f deploy.yaml
```

```
deployment.apps/fastapi-deployment unchanged
```

In [40]:
```
%%bash
kubectl get pods
```

```
NAME                                  READY   STATUS    RESTARTS   AGE
fastapi-deployment-86b66f9f84-7dkcs   1/1     Running   0          25m
fastapi-deployment-86b66f9f84-8x4tn   1/1     Running   0          25m
```

Манифест `deploy.yaml` применён в локальном Kubernetes-кластере Docker Desktop. Образ `hw4-fastapi-app:v1` был предварительно собран локально, для его использования внутри кластера задана политика `imagePullPolicy: Never`.

Результат:

NAME READY STATUS RESTARTS AGE

fastapi-deployment-86b66f9f84-7dkcs 1/1 Running 0 38s

fastapi-deployment-86b66f9f84-8x4tn 1/1 Running 0 38s

6. Итоговое оформление

В итоговых выводах дайте 5–8 предложений о своем опыте работы с инструментами модуля: что оказалось простым, что вызвало трудности, какие выводы сделали о применимости Docker/Kubernetes в реальных проектах.

Что я вынес из задания

Проще всего дался Docker - написал Dockerfile, собрал, запустил, проверил. Все довольно предсказуемо.

С docker-compose уже пришлось повозиться: YAML, сети, healthcheck'и, имена контейнеров - чуть ошибся, и связка не взлетает.

Kubernetes оказался самым сложным. Мало написать deploy.yaml - нужен живой кластер, доступный образ, правильный imagePullPolicy. Без понимания инфраструктуры легко не сделать задание.

Главный вывод: Docker отлично решает задачу "чтобы работало везде одинаково", а Kubernetes - "чтобы работало надежно и масштабировалось".

В реальных проектах они идут в связке, и это оправдано.